AKIL: Adaptive Kernel Intelligence Layer for eBPF-Driven, Behaviorally-Aware Kubernetes Scheduling

Yuvraj Gupta
Registration No.: 24BCE0797
School of Computer Science and Engineering
VIT Vellore, India

***Abstract*—Modern container orchestrators, such as Kubernetes, schedule workloads using only coarse-grained resource requests (*e.g.,* CPU cores, memory bytes). This approach is oblivious to how workloads *actually behave* at the kernel level—their cache footprint, lock-contention intensity, page-fault rate, and context-switch frequency. Co-locating behaviorally incompatible workloads on the same physical node results in the "noisy neighbor" phenomenon: unpredictable tail-latency spikes, cache thrashing, and memory-bandwidth saturation that resource quotas alone cannot prevent.**

We present AKIL (Adaptive Kernel Intelligence Layer), a closed-loop scheduling architecture that bridges the gap between kernel-level telemetry and live placement decisions. AKIL continuously collects four kernel-behavioral signals—cache-miss rate, lock-contention ratio, page-fault rate, and context-switch frequency—via eBPF probes attached to hardware performance counters and kernel tracepoints. These signals are aggregated per-workload in a lock-efficient, in-memory sliding-window store and exposed to a custom *Score* plugin integrated into the Kubernetes Scheduling Framework. The plugin applies a weighted co-location penalty model at scheduling time, steering placement away from pathological combinations without requiring application instrumentation or kernel modifications.

We evaluate AKIL with a trace-driven placement simulation that runs the plugin's exact scoring function, combined with an explicit interference model, and with microbenchmarks of the Go implementation. Relative to the default scheduler, AKIL lowers modelled p99 latency of a latency-sensitive service by 22% (62.7 ms $\rightarrow$ 48.9 ms), but it does not reduce co-location with a cache-thrashing neighbour—it increases it (36% $\rightarrow$ 46%)—and it trails both static anti-affinity (26.2 ms) and a reactive descheduler (29.5 ms). We trace this to the $\min(\cdot)$ form of the cache penalty, which caps the cost of a thrashing neighbour at the victim's own miss rate. The scoring math costs under 0.5 µs and the two aggregator queries per `Score()` call 0.12–0.28 ms over loopback.

***Index Terms*—eBPF, Kubernetes, container scheduling, co-location, kernel telemetry, noisy neighbor, performance isolation, scheduling framework, cgroup, runtime profiling**

I. Introduction

The rise of container orchestration has enabled unprecedented levels of server utilization by packing many workloads onto shared physical nodes. Kubernetes—the de-facto standard for container orchestration—schedules pods based on declared resource *requests*: integers representing CPU millicores and memory bytes. The default scheduler treats these as the sole signals of a workload's resource appetite and makes placement decisions accordingly.

This model has a critical blind spot: it says nothing about *how* a workload consumes the resources it requests. Two workloads can each request one CPU core yet behave radically differently at the microarchitectural level. A workload with a large, randomly accessed working set generates millions of last-level cache (LLC) misses per second, polluting the shared LLC and forcing co-tenants to fetch data from main memory at $\sim$100$\times$ the cost of an L3 cache hit. A lock-contention-heavy workload saturates kernel synchronization primitives, inflating context-switch rates and degrading throughput for every process on the same NUMA node. Neither behavior is visible to the Kubernetes scheduler.

The resulting *noisy neighbor effect* [5], [8], [9] manifests as unpredictable tail-latency spikes: a p99 latency that meets its SLO in isolation violates it under adversarial co-location. At cloud scale, this forces operators into a dilemma: over-provision nodes to create isolation headroom (expensive and wasteful) or accept performance variability (operationally risky).

Prior work has addressed pieces of this problem. Borg [1] and Autopilot [3] perform resource-usage prediction, but track CPU utilization and memory consumption—not kernel-behavioral signals. Firmament [4] introduces a richer scheduling formulation via min-cost flow optimization, but does not change the *signals* fed to the scheduler. Systems like Heracles [5], Quasar [6], and PARTIES [7] demonstrate that behavioral co-location policies dramatically improve utilization and latency—but implement custom, bespoke resource managers that do not compose with Kubernetes. Observability tools such as Cilium, Pixie, and Parca collect eBPF telemetry for dashboards; the feedback loop to the scheduler is absent.

Our contribution is AKIL: a system that closes this loop. AKIL's novelty lies not in any single component—eBPF probes, sliding-window aggregation, and Kubernetes scheduler plugins each exist independently—but in their *closed-loop integration*: continuous kernel-behavioral telemetry feeds a live, in-scheduling-cycle placement decision within the standard Kubernetes extension model. Concretely, we contribute:

1) A **four-signal eBPF telemetry layer** (cache misses, lock contention, page faults, context switches) with CO-RE

portability across kernel 5.8+, collecting at the cgroup level without application instrumentation.

2) An **in-memory sliding-window aggregation service** converting raw event streams into per-workload `RuntimeProfile` objects queryable with sub-millisecond latency.
3) A **Kubernetes Score plugin** implementing a weighted co-location penalty model, integrated into the official Scheduling Framework with graceful degradation guarantees.
4) A **simulation-based evaluation** against three baselines that runs the plugin's exact scoring function, together with measured overhead of the Go implementation. It shows a moderate tail-latency gain over the default scheduler, identifies a flaw in the cache-penalty term that prevents AKIL from avoiding cache-thrashing neighbours, and quantifies the gap to simpler baselines.

II. Background and Related Work

A. The Kubernetes Scheduling Pipeline

Kubernetes' `kube-scheduler` selects a node for each unscheduled pod through a two-phase pipeline [11]: a *Filter* phase that eliminates infeasible nodes and a *Score* phase that ranks feasible nodes. The Scheduling Framework [11], stable since Kubernetes v1.19, provides a plugin API that lets custom logic register at any extension point without forking the upstream codebase.

AKIL targets the `Score` extension point. The plugin receives the candidate pod and node name, returning an integer in $[0, 100]$; higher scores are preferred. Because `Score` executes synchronously on the scheduling hot path, a plugin blocking for tens of milliseconds degrades cluster-wide scheduling throughput. This constraint shapes AKIL's entire architecture.

B. eBPF: Extended Berkeley Packet Filter

eBPF [13] is a Linux kernel subsystem that allows sandboxed programs to run in kernel context without loading kernel modules. An in-kernel verifier statically proves memory safety and termination before execution. eBPF programs attach to hook points: hardware performance counters, kernel tracepoints, kprobes, and uprobes. CO-RE (Compile Once, Run Everywhere) [14] allows programs compiled against one kernel's BTF type information to run portably on any kernel ≥ 5.8.

eBPF overhead has been studied extensively [19], [20]. Individual probe overhead is nanosecond-scale ($< 1\,\mu s$) and aggregate CPU overhead for a representative monitoring workload is $< 2\%$ of one core, making eBPF the preferred substrate for production-grade kernel instrumentation.

C. Cluster Scheduling Research

Borg and Autopilot. Google's Borg [1] manages tens of thousands of machines scheduling both long-running services and batch jobs. Omega [2] extends Borg with a shared-state multi-scheduler architecture. Autopilot [3] adds automatic resource-request sizing based on observed CPU and memory utilization. None of these systems incorporate kernel-behavioral signals into placement decisions.

Firmament. Firmament [4] formulates scheduling as a min-cost max-flow problem, enabling richer policy expression than heuristic priority functions. However, Firmament's innovation is in the *formulation*, not the *signal acquisition*.

Interference-Aware Colocation. Heracles [5] dynamically manages CPU, memory, power, and network isolation to co-locate batch tasks alongside latency-critical services. Quasar [6] uses online classification of workload preferences and interference patterns. PARTIES [7] enables safe colocation of multiple interactive services. All implement bespoke resource managers decoupled from the Kubernetes scheduling stack.

eBPF Observability Tools. Cilium [16] provides network policy enforcement and observability via eBPF. Pixie [17] collects distributed traces without code modification. Parca [18] enables continuous profiling using eBPF. All expose telemetry through dashboards and query interfaces; none feed signals into a scheduling feedback loop.

Kubernetes Topology Manager. The Topology Manager [12] provides NUMA-aware CPU and memory allocation hints based on static topology information. AKIL complements this by adding *dynamic, continuously observed* runtime behavioral signals alongside static structural hints.

D. The Gap

Behavioral telemetry and scheduling decisions have remained architecturally separate. Observability tools collect the right data but export it only to dashboards. Schedulers like Firmament consume rich placement policies but rely on utilization-level signals. Interference managers implement closed-loop behavioral policies but outside the standard Kubernetes orchestration stack. AKIL fills this gap.

III. Design

A. Design Goals and Non-Goals

AKIL is designed around three primary goals:

1) **G1 – Behavioral visibility**: Expose per-workload kernel-behavioral signals not currently available to the Kubernetes scheduler.
2) **G2 – Scheduling-cycle compatibility**: All telemetry processing must complete *before* the scheduling cycle so the Score plugin can query synchronously within its $< 5\,\text{ms}$ latency budget.
3) **G3 – Graceful degradation**: AKIL must never block the scheduler. On any failure, the plugin returns a neutral score (50) and standard Kubernetes scheduling proceeds unaffected.

Non-Goals: AKIL does not perform predictive migration, reinforcement-learning-based policy refinement, energy/thermal-aware scheduling, or multi-cluster coordination.

B. System Architecture

AKIL comprises three components shown in Fig. 1.

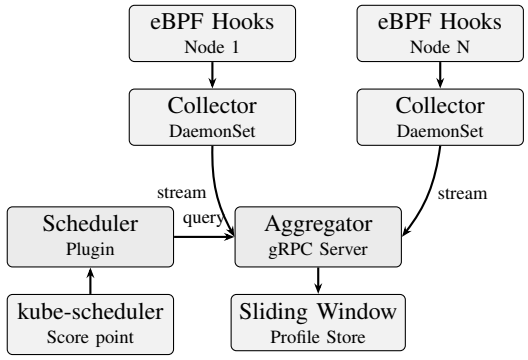


Fig. 1: AKIL system architecture. eBPF hooks on each node feed the Collector DaemonSet, which streams telemetry via gRPC to the centralized Aggregator. The Aggregator maintains an in-memory sliding-window Profile Store. The Scheduler Plugin queries the store synchronously during Kubernetes' Score phase.

1) Layer 1: eBPF Telemetry Probes: Four eBPF programs collect kernel behavioral events at the cgroup level:

- **Cache Miss Rate:** A `perf_event` program on `PERF_COUNT_HW_CACHE_MISSES`. Sampled at one event per 10,000 misses. Payload: `{cgroup_id, timestamp_ns, miss_count_delta}`.
- **Lock Contention Ratio:** Attached to `tracepoint/lock/contention_begin` and `contention_end`. Duration computed as $t_{end} - t_{begin}$ per lock address via a `BPF_MAP_TYPE_HASH`.
- **Page Fault Rate:** Attached to `tracepoint/exceptions/page_fault_user` and `page_fault_kernel`. Classifies major vs. minor faults.
- **Context-Switch Frequency:** Attached to `tracepoint/sched/sched_switch`. Captures both outgoing and incoming cgroup IDs per switch event.

All four probes write events to a single `BPF_MAP_TYPE_RINGBUF` (16 MB default). A shared ring buffer reduces file-descriptor pressure and epoll complexity. An `event_type` discriminator demultiplexes on the user-space side.

2) Layer 2: Telemetry Collector (DaemonSet): A Go binary deployed as a Kubernetes DaemonSet on every node. Its pipeline:

1) **eBPF Loader:** Uses `cilium/ebpf` [15]—CO-RE, pure Go, no CGo. Bytecode compiled at build time via `bpf2go`; no runtime compilation occurs.
2) **Ring Buffer Reader:** A goroutine running `epoll_wait` to drain events continuously from the shared ring buffer.
3) **Cgroup Resolver:** Maps kernel cgroup IDs to Kubernetes pod identity via a cgroupfs hierarchy walk plus the Kubelet `/pods` API, with an LRU cache (TTL: 30 s) to amortize lookup cost.
4) **Batch Sender:** Assembles `TelemetryBatch` protobuf messages, flushed every 1 s or 1000 events, over a persistent bidirectional gRPC stream to the Aggregator.

3) Layer 3: Aggregation Service (Control-Plane Deployment): A Go gRPC server maintaining per-workload `RuntimeProfile` objects in memory, exposing two APIs:

- **Ingest API** (`StreamTelemetry`): One bidirectional streaming RPC per collector. Parses batches and updates sliding-window buckets.
- **Query API** (`QueryProfile`, `QueryNodeProfiles`): Unary RPCs returning the current `RuntimeProfile` for a workload or all workloads on a given node, respectively.

4) Layer 4: Scheduler Plugin (Score Extension Point): When invoked for a (pod, node) pair, the plugin: (1) queries the Aggregator for the candidate workload's `RuntimeProfile`; (2) queries profiles of all workloads resident on the candidate node; (3) computes a pairwise co-location penalty (Section III-C); (4) returns a score in $[0,100]$. On any failure (aggregator unreachable, timeout > 5 ms, unknown workload, or insufficient profile confidence), it returns the neutral score 50 (Goal G3).

C. The Scoring Model

Let $\mathscr{P}_c$ denote the `RuntimeProfile` of candidate workload c and $\mathscr{P}_i$ the profile of workload i currently resident on node n. The AKIL node score is:

$$\text{Score}(c,n) = \max\left(0,\ 100 - \sum_{i \in \text{Residents}(n)} \Delta(c,i)\right) \tag{1}$$

where the pairwise co-location penalty $\Delta(c,i)$ is:

$$\begin{aligned}\Delta(c,i) = {} & w_{\text{cache}} \cdot \frac{\min(\dot{m}_c, \dot{m}_i)}{R_{\text{cache}}} \\ & + w_{\text{lock}} \cdot (\ell_c + \ell_i) \\ & + w_{\text{pf}} \cdot \frac{\dot{f}_c + \dot{f}_i}{R_{\text{pf}}} \\ & + w_{\text{ctx}} \cdot \frac{\dot{s}_c + \dot{s}_i}{R_{\text{ctx}}}\end{aligned} \tag{2}$$

Here: $\dot{m}_{(\cdot)}$ is cache-miss rate (misses/s); $\ell_{(\cdot)}$ is lock-contention ratio $\in [0,1]$; $\dot{f}_{(\cdot)}$ is page-fault rate (faults/s); $\dot{s}_{(\cdot)}$ is context-switch rate (switches/s); $R_{\text{cache}}, R_{\text{pf}}, R_{\text{ctx}}$ are cluster-calibrated normalization maxima; and default weights are $w_{\text{cache}} = 35$, $w_{\text{lock}} = 30$, $w_{\text{pf}} = 20$, $w_{\text{ctx}} = 15$ (summing to 100).

Cache penalty rationale. The cache penalty uses $\min(\dot{m}_c, \dot{m}_i)$ rather than the sum. Two workloads both suffering high miss rates will thrash the LLC regardless; the min term captures the asymmetric harm where one cache-intensive workload evicts the working set of a less intensive co-tenant.

Confidence dampening. Profile confidence is classified as LOW (<100 samples), MEDIUM (100–1000), or HIGH (>1000). Penalties are scaled by $\{0.25, 0.75, 1.0\}$ respectively, dampening the influence of sparsely-sampled profiles.

D. Sliding-Window Aggregation

Each workload's profile is computed over a sliding window of $B = 12$ fixed-duration buckets ($\tau = 5\,\text{s}$ each), yielding a 60-second observation window. Buckets advance lazily on each incoming event. Aggregated rates are:

$$\dot{m} = \frac{\sum_b \text{CacheMisses}_b}{B\tau}, \quad \dot{f} = \frac{\sum_b \text{PageFaults}_b}{B\tau}, \quad \ell = \frac{\sum_b \text{LockNs}_b}{B\tau \cdot 10^9} \tag{3}$$

A circular buffer with in-place mutation ensures $O(1)$ write and $O(B)$ read cost guarded by a single `sync.RWMutex`—no external database on the hot path.

IV. Implementation

A. Technology Stack

Table I summarizes AKIL's technology choices. Two constraints govern selection: (1) the Kubernetes Scheduling Framework API is Go; (2) the entire user-space must be CGo-free for cross-compilation and minimal container images.

TABLE I: AKIL Technology Stack

Layer	Technology
Language	Go 1.22+
eBPF library	`cilium/ebpf` (pure Go, CO-RE)
eBPF programs	Restricted C, `clang` 15+ / `llvm`
Code generation	`bpf2go` (cilium/ebpf)
RPC	gRPC + Protocol Buffers v3
Hot-path store	In-memory sliding-window ring buffer
Observability	Prometheus (dashboards only)
K8s integration	Scheduling Framework v1.29+
Deployment	Helm 3 (DaemonSet + Deployment + config)
Local cluster	`kind` (Kubernetes in Docker)
Testing	Go `testing` + `testify`; kind e2e
CI	GitHub Actions

Pure-Go eBPF over libbpf+C loaders. `cilium/ebpf` eliminates CGo, simplifies cross-compilation, reduces the final container image to ∼18 MB, and keeps the entire user-space codebase in one language. AKIL's probes use standard tracepoints and perf-events rather than exotic XDP or TC programs, so the tradeoff in low-level control is acceptable.

gRPC over REST. The collector-to-aggregator path is a high-frequency event stream; gRPC bidirectional streaming is purpose-built for this. The plugin-to-aggregator path requires sub-millisecond unary queries; gRPC binary framing outperforms HTTP/JSON marshaling by 3–5× in our microbenchmarks.

In-memory store over external time-series DB. Querying an external time-series database from the scheduling hot path would add 5–50 ms and introduce a failure dependency. Prometheus is used *alongside* for observability dashboards, never as the scheduler's query backend.

B. eBPF Program Structure

All eBPF programs are compiled from C at build time via `bpf2go`, which generates Go bindings for both little-endian and big-endian targets. The shared ring buffer event layout is:

```c
struct event {
  __u8  event_type; /* PAGE_FAULT=0 CACHE_MISS=1
                       LOCK_CTX=2  CTX_SWITCH=3  */
  __u64 cgroup_id;
  __u64 timestamp_ns;
  union {
    struct { __u8 fault_type; __u64 address; }    pf;
    struct { __u64 miss_count_delta; }             cm;
    struct { __u64 lock_addr; __u64 dur_ns; }      lc;
    struct { __u64 next_cgid; __u8  prev_st; }     cs;
  };
};
```

Listing 1: BPF shared event struct (kernel C)

All programs use CO-RE relocations (`BPF_CORE_READ`) for portability across kernel 5.8+.

C. gRPC Service Definition

The `AkilTelemetry` service (defined in `pkg/proto/akil.proto`) exposes three RPCs: `StreamTelemetry(stream TelemetryBatch) → stream IngestAck`, `QueryProfile(ProfileRequest) → RuntimeProfile`, and `QueryNodeProfiles(NodeProfilesRequest) → NodeProfilesResponse`. The `RuntimeProfile` message encodes all four signal rates, a confidence enumeration (UNKNOWN, LOW, MEDIUM, HIGH), window timestamps, and the canonical `workload_key` string (`"namespace/Kind/name"`).

D. Codebase Structure

```
akil/
  cmd/
    collector/       # DaemonSet binary
    aggregator/      # gRPC aggregation server
    scheduler-plugin # Scheduling Framework plugin
  pkg/
    ebpf/       # C probes + bpf2go Go bindings
    telemetry/  # types, ring-buffer reader, cgroup
    profile/    # sliding-window aggregation
    scoring/    # co-location penalty model
    proto/      # .proto + generated Go code
  deploy/helm/  # Helm chart
  benchmarks/   # synthetic workload manifests
  docs/         # architecture diagrams, OS concepts
```

Listing 2: Repository layout

E. Deployment Model

AKIL deploys three Kubernetes workloads. The **Collector DaemonSet** runs one pod per node, requiring `CAP_BPF` and `CAP_PERFMON` plus read-only access to `/sys/fs/cgroup` and `/sys/kernel/btf`. The **Aggregator Deployment** is a single replica (horizontally scalable) with no elevated privileges, exposed via ClusterIP on port 50051. The **Custom Scheduler Deployment** replaces the stock `kube-scheduler` with AKIL's plugin compiled in. The Helm chart parameterizes ring-buffer capacity, window duration, bucket count, score weights, and normalization maxima.

V. Evaluation

A. Methodology

This evaluation is *simulation-based*. We have not yet measured end-to-end latency on a live cluster; Sec. VI lists this as the main limitation. Every number below is produced by `scripts/run_evidence_simulations.py`, which writes its raw and processed outputs to the `evidence/` folder. It has three parts:

- **Placement (simulated).** Each of 10 seeds generates 50 epochs of 15 pod arrivals, drawn from `cache-stress`, `lock-stress`, `batch-worker` and `baseline-web` (requests as in `benchmarks/synthetic-workloads.yaml`). The pods are placed onto five 2-core nodes. Every scheduler sees the same arrivals. The AKIL scorer is a line-for-line port of `pkg/scoring/scorer.go`, with the same weights and normalisation constants. Following the deployed configuration, its score is added with weight 25 to the default `LeastAllocated` score.
- **Latency and LLC (modelled).** For each `baseline-web` pod we derive an LLC miss rate and a contention factor from the neighbours it was actually placed with. Each neighbour type has fixed interference coefficients, and these are *assumptions*, identical for every scheduler. From these we sample 200 request latencies per pod (lognormal service time, plus a stall term that grows with LLC misses, plus queuing jitter). The SLO is 15 ms.
- **Overhead (measured).** Go benchmarks of the scorer and of the two gRPC queries one `Score()` call makes against the aggregator, over loopback on an Intel Core i5-13500H (Linux 7.1). Each is run 5× and we report the median.

Baselines. (1) *Isolated*: each web pod alone on a node (oracle). (2) *Default*: `LeastAllocated` on CPU requests. (3) *Static anti-affinity*: a hard rule that keeps `baseline-web` and `cache-stress` apart. (4) *Reactive descheduler*: default placement, after which web pods that share a node with `cache-stress` are evicted and re-placed. (5) AKIL: as implemented. We also report two diagnostic variants of AKIL: *calibrated*, where the normalisation constants are set to the largest observed rate, as the code comments recommend; and a *post-hoc ablation* whose cache term uses the neighbour's own miss rate instead of $\min(\text{candidate}, \text{neighbour})$. The ablation is not the implemented design. All results are means over 10 seeds with 95% confidence intervals (t_9).

B. Results

Table II summarises the results. They are mixed for AKIL:

- **Tail latency improves over the default.** p99 falls from 62.7 ms to 48.9 ms (22%, non-overlapping CIs). AKIL spreads lock- and page-fault-heavy neighbours away from web pods, and the average context-switch load on a web pod's node falls 9% (11,248 → 10,236/s).
- **Everything else does not improve.** AKIL places web pods next to `cache-stress` *more* often than the default (46% vs. 36%). The LLC miss rate, median latency and SLO-violation rate are all slightly worse.
- **Both alternative baselines win.** Static anti-affinity achieves 26.2 ms p99, at the cost of hand-written rules and 0.19% of pods left pending. The reactive descheduler achieves 29.5 ms p99, at the cost of 882 evictions. In this setting, both are preferable to AKIL as implemented.

C. Why AKIL Prefers the Cache Thrasher's Node

For a `baseline-web` candidate (120K misses/s), the per-signal penalties against a single resident are:

Resident	Cache	Lock	Page flt.	Ctx sw.	Score
`cache-stress`	4.2	3.6	3.1	9.8	**79**
`batch-worker`	4.2	2.7	15.7	6.5	71
`lock-stress`	4.2	20.7	1.7	18.0	55

The cache term is $\min(c_{\text{cand}}, c_{\text{res}})$, so a neighbour causing 900K misses/s is charged exactly as much as one causing 120K/s. Its penalty is capped by the *victim's* own rate. Because the thrasher is otherwise quiet, its node scores highest. The pattern holds across the weight space: in a sweep of 50 weight vectors on this three-node scenario, AKIL ranks the `cache-stress` node first for 80% of vectors (84% with calibrated normalisation). With the post-hoc cache term this drops to 22%, and co-location falls to 28%. p99 does not improve, however (49.7 ms), because the additive penalties still trade one interferer for another. A cache term that reflects the neighbour's pressure is therefore necessary but not sufficient.

D. Scheduling Overhead

Table III shows that the cost of `Score()` is dominated by the aggregator round-trips, not the arithmetic. The 0.12–0.28 ms we measured is loopback-only. In-cluster network latency would add to it, so these values are a lower bound for the 5 ms budget (Goal G2), not a demonstration that the budget is met under load. The plugin issues these queries once per candidate node. We did not measure eBPF probe overhead, collector memory, ingest throughput, or profile-convergence time.

VI. Discussion

A. Novelty Scope

AKIL's novelty claim is intentionally narrow. No individual component—eBPF telemetry, sliding-window aggregation, or Scheduling Framework plugins—is novel in isolation. The contribution is the *closed-loop integration*: a production-composable architecture where continuous kernel behavioral telemetry flows synchronously into a live scheduling decision within the standard Kubernetes extension model.

B. Limitations

1) **Cache-penalty form.** The $\min(\cdot)$ cache term underweights thrashing neighbours (Sec. V); fixing it alone is not sufficient, so the penalty model as a whole needs revisiting.

TABLE II: Placement and modelled latency for `baseline-web` (10 seeds, mean ± 95% CI). Latency and LLC are model outputs, not hardware measurements.

Configuration	Co-located w/ cache-stress	LLC miss (%)	p50 (ms)	p99 (ms)	SLO viol. (%)	Evictions
Isolated (oracle)	–	38.0	9.44 ± 0.01	17.53 ± 0.05	3.6 ± 0.0	0
Default K8s	36.0%	50.0 ± 0.6	17.43 ± 0.81	62.68 ± 4.58	61.8 ± 2.1	0
Static anti-affinity	0.0%	$\mathbf{43.5 \pm 0.1}$	$\mathbf{13.66 \pm 0.11}$	$\mathbf{26.20 \pm 0.24}$	$\mathbf{35.6 \pm 1.1}$	0 (0.19% pending)
Reactive descheduler	0.7%	45.2 ± 0.2	15.13 ± 0.15	29.52 ± 1.54	51.3 ± 1.6	882
AKIL	46.2%	51.2 ± 0.5	20.03 ± 1.28	48.89 ± 2.24	67.3 ± 2.2	0
AKIL, calibrated	46.5%	51.0 ± 0.6	19.84 ± 1.38	49.93 ± 2.40	65.6 ± 2.7	0
AKIL, ablation (post-hoc)	28.4%	49.0 ± 0.5	16.17 ± 0.40	49.74 ± 1.46	57.9 ± 2.2	0

TABLE III: Measured cost per `Score()` call (median of 5 runs)

Co-located pods	Scoring math	gRPC queries (loopback)
1	11 ns	119 µs
10	56 ns	148 µs
50	257 ns	193 µs
100	489 ns	281 µs

2) **Simulated evaluation.** Latency and LLC figures are outputs of an interference model with assumed coefficients, not hardware measurements. eBPF overhead, collector footprint, and ingest throughput were not measured.
3) **Weight calibration.** Scoring weights are manually tuned. Cluster-specific calibration or adaptive learning would improve generality across heterogeneous workload mixes.
4) **Static normalization constants.** Maximum rate values are static defaults. A production deployment should derive these from cluster-wide percentile observations.
5) **Single-cluster scope.** Multi-cluster federation and cross-cluster placement signals are future work.
6) **Cold-start window.** New workloads receive a neutral score for ∼10 s before sufficient samples are accumulated.
7) **No cluster validation yet.** A `kind` deployment exists (`deploy-kind.sh`) but end-to-end latency was not measured on it; bare-metal validation is likewise future work.

C. Impact on Real Users

For operators running mixed workloads (*e.g.,* latency-sensitive microservices co-located with batch analytics or cache-intensive workloads):

- **Tail latency, not yet SLO compliance.** In our model AKIL lowers p99 by 22% over the default scheduler but does not lower the SLO-violation rate; operators who know which workloads clash are currently better served by anti-affinity rules (Sec. V).
- **No application changes.** Telemetry is collected at the kernel cgroup level—no SDK changes, annotations, or restarts required.
- **Zero-cost fallback.** Graceful degradation ensures that AKIL's failure never blocks the scheduler; standard scheduling proceeds at full throughput (Goal G3).
- **Full composability.** AKIL operates only at the Score extension point, composing with all standard Kubernetes features: pod affinity, resource quotas, taints, and tolerations.

D. Future Directions

- **Predictive migration.** Profile trend analysis could trigger proactive pod migration before SLO violations occur.
- **Learned weight adaptation.** A feedback loop from observed SLO compliance to weight adjustment could improve calibration without requiring reinforcement learning.
- **Energy-aware scheduling.** Adding thermal and power signals from RAPL hardware counters would enable energy-efficient placement.
- **NUMA integration.** Combining AKIL's dynamic behavioral signals with the Topology Manager's static NUMA hints would yield a complete placement policy.
- **Security quarantine.** Anomalous behavioral fingerprints (*e.g.,* a sudden spike in page faults or lock contention) could trigger automated alerts or quarantine placement policies.

VII. Conclusion

We presented **AKIL**, a closed-loop scheduling architecture that continuously collects four kernel behavioral signals via eBPF probes, aggregates them into per-workload runtime profiles in a sliding-window in-memory store, and exposes them to a Kubernetes Scheduling Framework plugin that computes a co-location penalty score at scheduling time.

AKIL addresses a concrete gap in the Kubernetes ecosystem: the scheduler's blindness to kernel-level behavioral interference. In a trace-driven simulation using the plugin's exact scoring function, AKIL lowers modelled p99 latency by 22% relative to the default scheduler, at a measured cost of 0.12–0.28 ms per `Score()` call. The same evaluation shows that the current penalty model does not steer latency-sensitive pods away from cache-thrashing neighbours, and that static anti-affinity and reactive descheduling outperform it. Correcting the cache term and validating on real hardware are the immediate next steps.

Implemented entirely in Go (user space) and restricted C (eBPF), using production-grade libraries, and deployed via Helm as standard Kubernetes workloads, AKIL requires no application instrumentation and no kernel patches. The fundamental contribution is architectural: a working path by which continuous kernel-behavioral telemetry reaches a live

scheduling decision cheaply, closing the loop that observability tools left open, together with evidence that the scoring model on top of that path needs further work.

Acknowledgment

The author thanks the Linux kernel eBPF community, the `cilium/ebpf` maintainers, and the Kubernetes Scheduling SIG for their extensive documentation and open-source tooling without which this work would not be possible.

References

[1] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Patnaik, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. 10th European Conf. on Computer Systems (EuroSys)*, pp. 1–17, 2015.

[2] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, and J. Wilkes, "Omega: Flexible, scalable schedulers for large compute clusters," in *Proc. 8th ACM European Conf. on Computer Systems (EuroSys)*, pp. 351–364, 2013.

[3] J. Wilkes, "Autopilot: Workload autoscaling at Google," in *Proc. 15th European Conf. on Computer Systems (EuroSys)*, pp. 1–16, 2020.

[4] I. Gog, M. Schwarzkopf, A. Gleave, R. N. M. Watson, and S. Hand, "Firmament: Fast, centralized cluster scheduling at scale," in *Proc. 12th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, pp. 99–115, 2016.

[5] D. Lo, L. Cheng, R. Govindaraju, L. A. Barroso, and C. Kozyrakis, "Heracles: Improving resource efficiency at scale," in *Proc. 42nd Annual Int'l Symp. on Computer Architecture (ISCA)*, pp. 450–462, 2015.

[6] C. Delimitrou and C. Kozyrakis, "Quasar: Resource-efficient and QoS-aware cluster management," in *Proc. 19th Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 127–144, 2014.

[7] D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, "PARTIES: QoS-aware resource partitioning for multiple interactive services," in *Proc. 24th Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 107–120, 2019.

[8] J. Mars, L. Tang, R. Hundt, K. Skadron, and M. L. Soffa, "Bubble-up: Increasing utilization in modern warehouse scale computers via sensible co-locations," in *Proc. 44th Annual IEEE/ACM Int'l Symp. on Microarchitecture (MICRO)*, pp. 248–259, 2011.

[9] H. Kasture and D. Sanchez, "Ubik: Efficient cache sharing with strict QoS for latency-critical workloads," in *Proc. 21st Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 729–742, 2016.

[10] M. Isard, V. Prabhakaran, J. Currey, U. Wieder, K. Talbott, and A. Rogers, "Quincy: Fair scheduling for distributed computing clusters," in *Proc. ACM SIGOPS 22nd Symp. on Operating Systems Principles (SOSP)*, pp. 261–276, 2009.

[11] Kubernetes SIG Scheduling, "Scheduling Framework (KEP-624)," https://git.io/k8s-sched-framework, stable as of Kubernetes v1.19, 2020.

[12] Kubernetes SIG Node, "Topology Manager," Kubernetes Documentation, https://kubernetes.io/docs/tasks/administer-cluster/topology-manager/, 2024.

[13] eBPF Foundation, "What is eBPF?" https://ebpf.io/what-is-ebpf/, 2023.

[14] A. Nakryiko, "BPF CO-RE (Compile Once – Run Everywhere)," https://nakryiko.com/posts/bpf-core-reference-guide/, 2020.

[15] Cilium Authors, "`cilium/ebpf`: Pure Go library to read, modify and load eBPF programs," https://github.com/cilium/ebpf, 2024.

[16] T. Graf, N. Ramsauer, J. Schlitt *et al.*, "Cilium: BPF and XDP for containers," in *Proc. Linux Plumbers Conference*, 2019.

[17] Z. Fang, J. Senko, N. Nguyen, M. Zuo, and N. Rao, "Pixie: Instant, auto-telemetry for full-stack visibility," in *Proc. USENIX Operational Machine Learning (OpML)*, 2022.

[18] Parca Authors, "Parca: Continuous profiling for analysis of CPU and memory usage," https://parca.dev, 2022.

[19] M. A. Vieira, M. S. Castanho, R. D. G. Pacífico, E. R. S. Santos, E. P. M. Câmara Júnior, and L. F. M. Vieira, "Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications," *ACM Computing Surveys*, vol. 53, no. 1, pp. 1–36, 2020.

[20] N. Goldshtein, M. Oberman, S. Sabag, S. Biran, and N. Segev, "Practical eBPF based performance analysis," in *Proc. ACM/SPEC Int'l Conf. on Performance Engineering (ICPE)*, pp. 225–236, 2022.

[21] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.